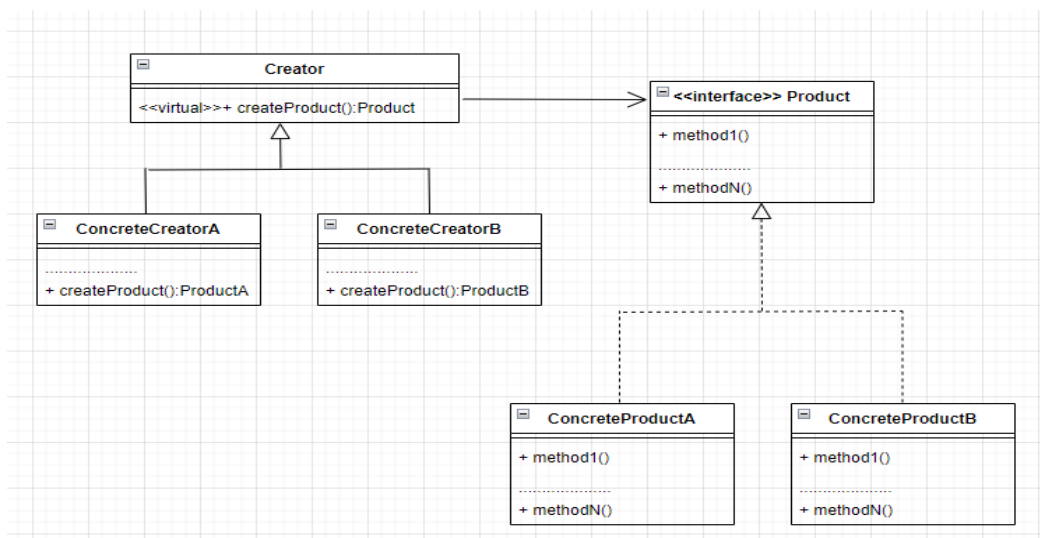


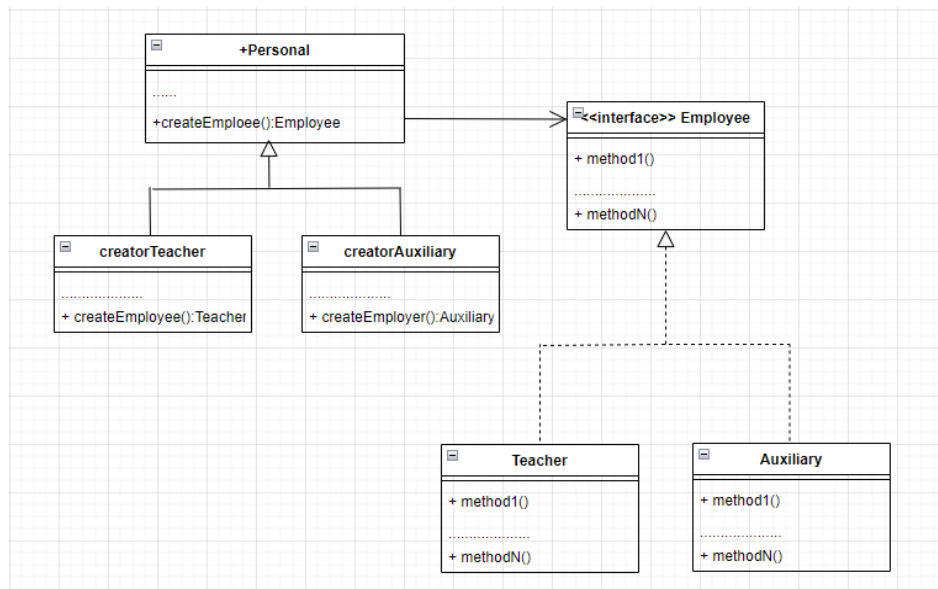
Порождающие паттерны

1. **Порождающие паттерны:** создание семейств новых объектов.
2. **Порождающие паттерны:** Factory Method (Фабричный метод), Abstract Factory (Абстрактная фабрика), Builder (Строитель), Prototype (Прототип), Singleton (Одиночка).

Factory Method

3. **Factory Method:** общий интерфейс для создания объектов в базовом классе, позволяя производному классу изменять тип создаваемых объектов. Фабрика классов COM (IClassFactory).
4. **Описание проблемы:** необходимо создавать объекты различного типа, в дальнейшем могут появиться другие типы объектов.
5. **UML-диаграмма:** **Product** – интерфейс создаваемых объектов, **ConcreteProduct** – различные продукты, **Creator** – создатель объектов, **ConcreteCreator** –реализация виртуального метода CreateProduct()





6. C#

```

public class Personal
{
    //.....
    public virtual IEmployee CreateEmployee( string name, string pass, DateTime start)
    {
        return new Auxiliary("", "", "", "", DateTime.Now); // по умолчанию
    }
    //.....
}
  
```

```

public class CreatorTeacher : Personal
{
    string department;
    string academdegree;
    string degree;
    string jobtitle;
    public CreatorTeacher(string department, string academdegree, string degree, string jobtitle)
    {
        this.department = department;
        this.academdegree = academdegree;
        this.degree = degree;
        this.jobtitle = jobtitle;
    }
    public override Teacher CreateEmployee(string name, string pass, DateTime start)
    {
        return new Teacher(department, academdegree, degree, jobtitle,
            name, pass, start);
    }
}
  
```

```

public class CreatorAuxiliary : Personal
{
    string department;
    string jobtitle;
    public CreatorAuxiliary(string department, string jobtitle)
    {
        this.department = department;
        this.jobtitle = jobtitle;
    }
    public override Auxiliary CreateEmployee(string name, string pass, DateTime start)
    {
        return new Auxiliary(department, jobtitle, name, pass, start);
    }
}
  
```

```
public interface IEmployee
{
    //.....
    int calcLengthNextVacation(); // вычисление продолжительности очередного отпуска
    //.....
}
```

```
public class Teacher : IEmployee
{
    public Teacher (string department, string academdegree, string degree, string jobtitle,
                    string name, string pass, DateTime start)
    { }
    public int calcLengthNextVacation()
    {
        int rc = 57;
        // rc =
        return rc;
    }
}
```

```
public class Auxiliary: IEmployee
{
    public Auxiliary(string department, string jobtitle,
                    string name, string pass, DateTime start)
    { }
    public int calcLengthNextVacation()
    {
        int rc = 24;
        // rc =
        return rc;
    }
}
```

```
public class CalcVacation
{
    //.....
    static public int calcLengthNextVacation(IEmployee employee)
    {
        return employee.calcLengthNextVacation();
    }
    //.....
}
```

```
static void Main(string[] args)
{
    CreatorAuxiliary crA = new CreatorAuxiliary("бухгалтерия", "кассир");
    CreatorTeacher crT = new CreatorTeacher("ИСИТ", "к.т.н.", "доцент", "доцент");

    Auxiliary a01 = crA.CreateEmployee("Петров П.П.", "mp222333", DateTime.Now);
    Teacher t01 = crT.CreateEmployee("Сидоров С.С.", "mp333444", DateTime.Now);

    int day_a01 = CalcVacation.calcLengthNextVacation(a01);
    Console.WriteLine(string.Format("Auxiliary: отпуск {0} дней", day_a01));
    int day_t01 = CalcVacation.calcLengthNextVacation(t01);
    Console.WriteLine(string.Format("Teacher: отпуск {0} дней", day_t01));

    Person[] persons = {crA, crT};

    IEmployee emp1 = persons[0].CreateEmployee("Сергеев С.С.", "mp444555", DateTime.Now); //бухгалтерия, кассир
    IEmployee emp2 = persons[1].CreateEmployee("Михайлов М.М.", "mp555666", DateTime.Now); // ИСИТ, к.т.н., доцент, доцент

    Console.WriteLine(string.Format("отпуск {0} дней", emp1.calcLengthNextVacation()));
    Console.WriteLine(string.Format("отпуск {0} дней", emp2.calcLengthNextVacation()));
}
```

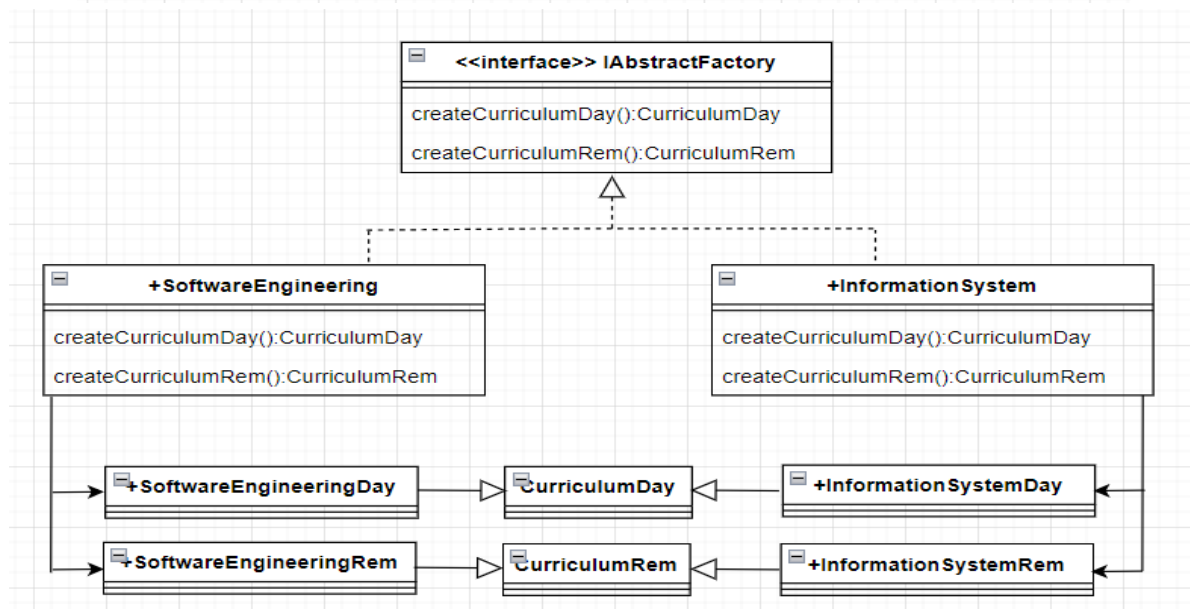
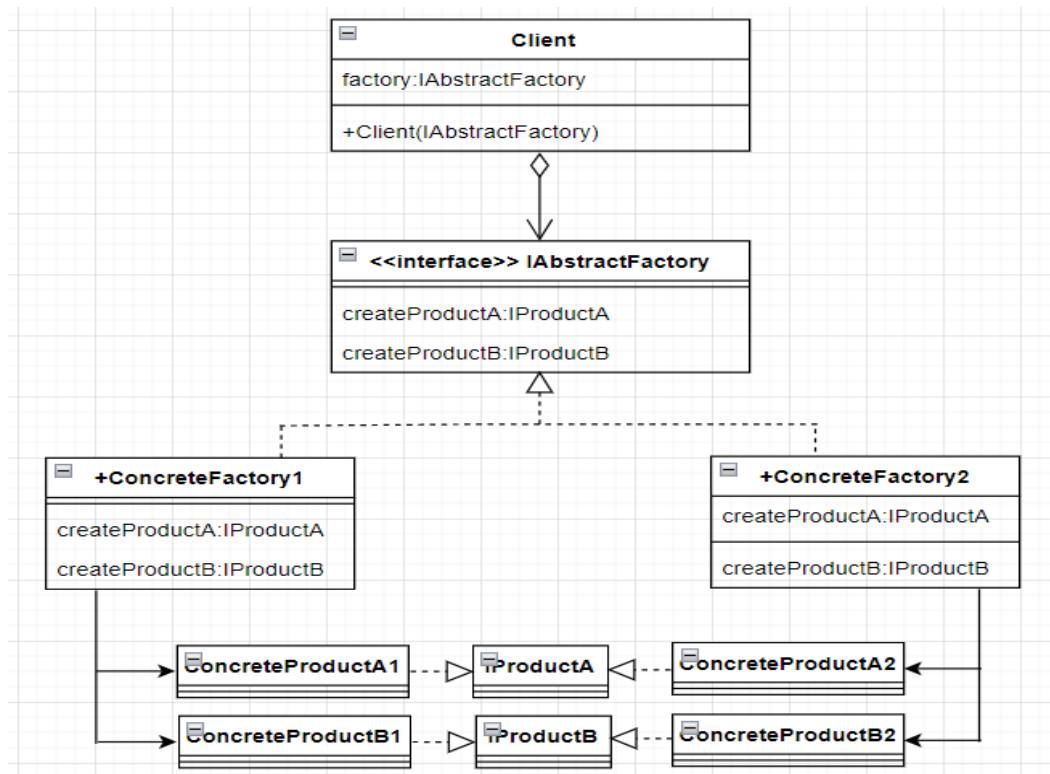
Консоль отладки Microsoft Visual Studio

```
Auxiliary: отпуск 24 дней
Teacher: отпуск 57 дней
отпуск 24 дней
отпуск 57 дней
```

Abstract Factory

7. **Abstract Factory:** порождающий паттерн для создания семейств связанных объектов.

8. **Abstract Factory/UML:**



9. C#

```

public interface IAbstractFactory
{
    CurriculumDay createCurriculumDay();
    CurriculumRem createCurriculumRem();
}
  
```

```

public class SoftwareEngineering : IAbstractFactory
{
    public string Name {get; set;} = "SoftwareEngineering";
    public CurriculumDay createCurriculumDay()
    {
        SoftwareEngineeringDay rc = new SoftwareEngineeringDay(this.Name);
        // rc = 
        return rc;
    }

    public CurriculumRem createCurriculumRem()
    {
        SoftwareEngineeringRem rc = new SoftwareEngineeringRem(this.Name);
        // rc = 
        return rc;
    }
}
  
```

```

public class InformationSystem:IAbstractFactory
{
    public string Name { get; set; } = "Information System";
    public CurriculumDay createCurriculumDay()
    {
        InformationSystemmgDay rc = new InformationSystemmgDay(this.Name);
        // rc =
        return rc;
    }

    public CurriculumRem createCurriculumRem()
    {
        InformationSystemRem rc = new InformationSystemRem(this.Name);
        // rc =
        return rc;
    }
}

```

```

public class SoftwareEngineeringDay:CurriculumDay
{
    public override string Name { get; set;}
    public SoftwareEngineeringDay(string name)
    {
        this.Name = name;
    }
}
public class SoftwareEngineeringRem: CurriculumRem
{
    public override string Name { get; set; }
    public SoftwareEngineeringRem(string name)
    {
        this.Name = name;
    }
}

```

```

public class InformationSystemmgDay : CurriculumDay
{
    public override string Name { get; set;}
    public InformationSystemmgDay(string name)
    {
        this.Name = name;
    }
}
public class InformationSystemRem : CurriculumRem
{
    public override string Name { get; set;}
    public InformationSystemRem(string name)
    {
        this.Name = name;
    }
}

```

```

public class CurriculumDay
{
    public string Regim { get; set; } = "Day";
    public virtual string Name { get; set; }
}
public class CurriculumRem
{
    public string Regim { get; set; } = "Remote";
    public virtual string Name { get; set;}
}

```

```

public class Department
{
    IAbstractFactory[] factories;
    public Department(params IAbstractFactory[] factories)
    {
        this.factories = factories;
    }
    public void print()
    {
        foreach(var f in factories)
        {
            CurriculumDay day = f.createCurriculumDay();
            CurriculumRem rem = f.createCurriculumRem();
            Console.WriteLine(string.Format("Специальность {0},режим {1}", day.Name, day.Regim));
            Console.WriteLine(string.Format("Специальность {0},режим {1}", rem.Name, rem.Regim));
        }
    }
}

```

```
SoftwareEngineering seng = new SoftwareEngineering();  
InformationSystem isys = new InformationSystem();  
Department department = new Department(seng, isys);  
department.print();
```

Консоль отладки Microsoft Visual Studio

```
Специальность SoftwareEngineering,режим Day  
Специальность SoftwareEngineering,режим Remote  
Специальность Information System,режим Day  
Специальность Information System,режим Remote
```

10. КОНЕЦ